A PROJECT REPORT

on

“AI SHOPPING AGENT”

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR’S DEGREE IN
INFORMATION TECHNOLOGY

BY

Ridhi Kumari
23052828

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024

August 2026

CERTIFICATE

This is to certify that the project entitled

"AI SHOPPING AGENT"

submitted by

Ridhi Kumari

23052828

is a record of bona fide work carried out by the student, in partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering OR Information Technology) at KIIT Deemed to be University, Bhubaneswar. This work was completed during the year 2025-2026.

Date: / /

Signature of Ridhi Kumari

ABSTRACT

AI Shopping Agent is a web-based conversational commerce system that converts a natural-language buying intent into a small, defensible set of purchasable products drawn from live cross-merchant retail inventory. The system asks a clarifying question only when a missing constraint would materially change the result set, searches the live Shopify catalogue through the Model Context Protocol, refines or paginates results as the shopper reacts, compares products and sellers, verifies research claims against web evidence, and presents a direct merchant purchase route once an item is chosen.

The distinguishing property of the system is architectural rather than conversational. Product cards, comparison tables and seller listings are rendered exclusively from tool results, so there is no code path by which model-generated prose can become a product claim. Fabricating a product, a price or a specification is therefore not a behaviour the model is merely discouraged from, but one the architecture does not permit.

The application combines a Next.js and React frontend, a TypeScript agent service, Shopify Catalog MCP integration, streaming model output through the Vercel AI SDK, OpenRouter and DeepSeek model access, PostgreSQL persistence and Redis caching. Verification is layered: backend unit tests, static type checking, browser-driven end-to-end tests, and an eight-scenario agent evaluation harness that asserts on the agent's sequence of tool calls rather than on the readability of its prose. That harness identified two failures whose on-screen output appeared entirely correct, which is the central empirical finding reported here.

Keywords: Agentic AI, agentic commerce, Model Context Protocol, tool calling, grounded generative UI, product recommendation, behavioural evaluation, Next.js, conversational shopping.

Contents

List of Figures

Chapter 1

Introduction

Online product discovery remains difficult whenever a buyer's need is expressed as an intent rather than as an exact product name. Conventional keyword search returns long result lists, forces the buyer to translate requirements into filters, and makes cross-merchant price and seller comparison tedious. A shopper who knows only that they want “something for editing 4K video on a budget” must convert that description into keywords, then into filters, then compare a dozen browser tabs by hand. The translation step is where most shoppers abandon the task.

Fast-moving categories add a second risk. A technically matching result may be several hardware generations old, poorly rated, unavailable, or drawn from an unrelated product class such as an accessory rather than the device itself. Keyword relevance and real-world usefulness diverge sharply in exactly the categories where buyers most need help.

AI Shopping Agent addresses this gap with a conversational workflow grounded in live catalogue data. The agent clarifies only decisive missing constraints, searches the live cross-merchant catalogue, refines or paginates results as the conversation develops, compares two to four products, compares sellers for a chosen product, verifies research claims through web evidence, and emits a checkout call to action only after the buyer has selected an item. Structured product cards and comparison components are generated from tool outputs, which removes the opportunity for fabricated prices, sellers, specifications or links.

1.1 Motivation for an Agentic Approach

A single model call is sufficient for tasks whose shape is known in advance. Product discovery is not such a task, and three properties of the problem justify the additional complexity of an agent architecture.

- Unknown depth. The number of steps required cannot be determined before the conversation begins. A precise request such as a named product may need one search, whereas an open-ended request may require a clarification, a search, a refinement and a comparison before a recommendation is possible.
- External truth. The answer resides outside the model, in a catalogue that changes continuously. The model's own parametric knowledge is not merely insufficient here, it is actively harmful: a confidently recalled but outdated price is worse for the buyer than no price at all.
- Recoverable failure. Catalogue searches return empty or off-target results frequently. A system that can observe this condition and re-query with altered terms outperforms one that simply reports failure to the user.

1.2 Objectives

The project set out to achieve the following objectives.

1. Translate natural-language buying intent into grounded product recommendations drawn from live cross-merchant inventory, without fabricated catalogue facts.
2. Implement a bounded multi-step reasoning loop with explicit, inspectable stopping conditions rather than open-ended generation.
3. Design a typed tool interface in which each capability has a narrow responsibility and a validated input schema.
4. Maintain conversational working memory so that follow-up requests such as "cheaper" or "show me more" resolve against the previous search.
5. Enforce factual grounding structurally, through an interface contract, rather than through prompt instructions alone.
6. Evaluate the resulting system on its behaviour, by asserting on the sequence of tool calls it produces, and report the outcome honestly including failures.

1.3 Organisation of the Report

Chapter 2 introduces the technical foundations on which the project rests: the tool-calling agent loop, the Model Context Protocol, grounded generative interfaces, and the evaluation of agent behaviour. Chapter 3 states the problem formally, records the planning approach, presents the requirement specification, and describes the system design including its architecture, request lifecycle and data model. Chapter 4 documents the implementation, the verification plan, the results obtained and the quality-assurance mechanisms. Chapter 5 records the design, coding and testing standards adopted. Chapter 6 concludes and sets out the future scope. Figure 1.1 shows the implemented chat surface used to begin a shopping conversation.

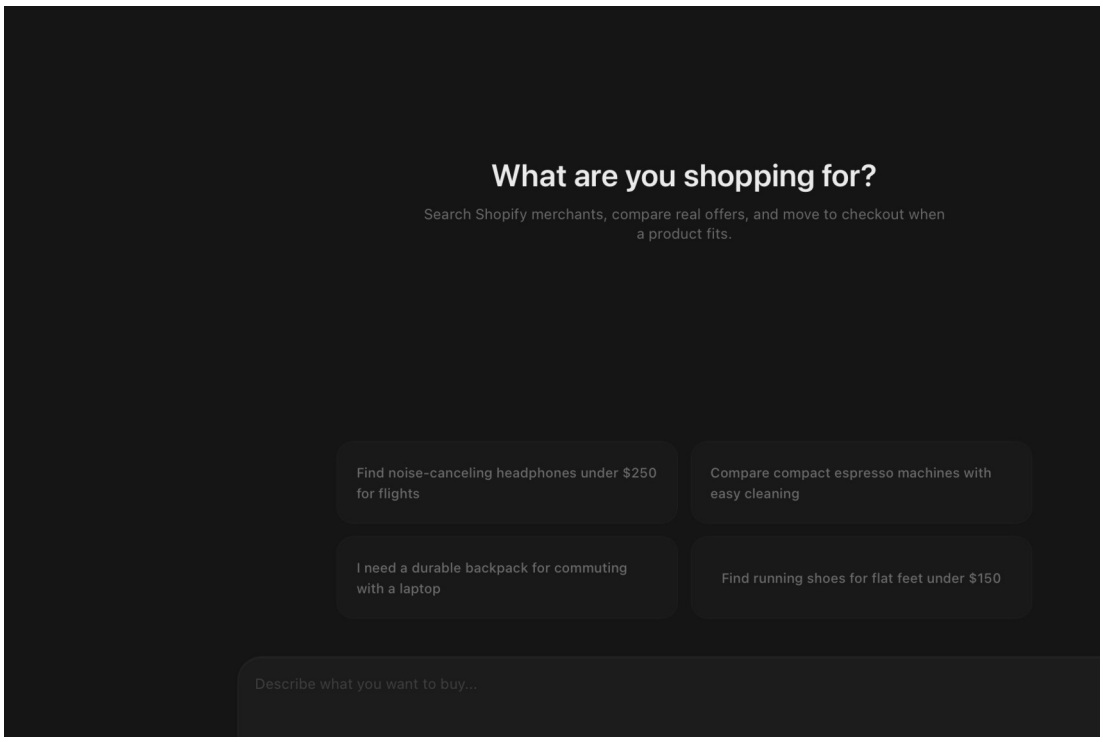

Figure 1.1: Shopping Agent chat interface

Chapter 2

Basic Concepts / Literature Review

The project draws on four areas that have converged only recently: agentic artificial intelligence and tool calling, structured protocols for exposing external systems to agents, generative user interfaces, and the evaluation of autonomous behaviour. This chapter reviews each in turn and relates it to the design decisions taken later in the report.

2.1 Agentic AI and the Tool-Calling Loop

An agentic system differs from a single model invocation in that the model is permitted to choose actions, observe their results, and choose again. The canonical loop consists of four repeating stages: the model reasons over the conversation and any results gathered so far; it selects an action from a declared set of tools; the runtime executes that action and returns a typed result; and the result is appended to the context for the next iteration. The loop terminates when a stopping condition is satisfied.

The literature identifies a standard set of capabilities against which such systems are assessed. Tool use is the ability to invoke external functions with validated arguments. Multi-step reasoning is the ability to chain those invocations, with each step informed by the previous result. Planning is the policy that determines which action to take first for a given input. Working memory carries state across turns so that later requests can refer to earlier ones. Self-correction is the ability to detect an unsatisfactory result and retry differently. Grounding constrains claims to evidence actually retrieved.

Stopping conditions deserve particular attention because they are frequently neglected. An agent without a bound on its own iteration can loop indefinitely on a failing action, consuming budget and producing nothing. Two categories of stopping condition are used in this project: a numeric budget on the number of reasoning steps permitted per turn, and a semantic condition that terminates the turn when a specific action occurs. The second category is the more interesting, and is discussed in Section 4.1.

2.2 The Model Context Protocol and Agentic Commerce

The Model Context Protocol (MCP) is an emerging standard for exposing external systems to language-model agents in a structured, discoverable way. Rather than requiring each application to invent a bespoke integration, MCP defines a transport and an envelope in which a client invokes named operations on a server and receives structured content in reply. Requests are carried as JSON-RPC messages, and the calling agent may declare its own identity and capabilities as part of the request metadata.

Agentic commerce applies this pattern to retail. A cross-merchant catalogue exposed through MCP allows an agent to search inventory spanning many independent merchants, retrieve normalised product and variant records, and obtain genuine purchase routes for a chosen item. This is materially different from integrating a single storefront: the agent can compare

sellers of the same product and recommend across the whole network rather than within one merchant's stock.

Three operations are relevant to this project: a keyword search accepting filters and pagination, a detail retrieval for a single product identifier, and a batch retrieval used as a fallback when detail retrieval fails. The catalogue returns products, variants, sellers, price ranges, media, specifications, availability and merchant links, all of which can serve as evidence for a recommendation. An important operational constraint accompanies this access: the catalogue's terms prohibit caching of search results and catalogue images, which rules out an obvious performance optimisation and is discussed in Section 3.3.4.

2.3 Grounded Generative User Interfaces

A generative user interface is one whose components are produced in response to model output rather than being fixed in advance. The naive implementation permits the model to emit markup or prose that the interface renders directly. This is convenient and unsafe: the model may invent a price, a specification or an entire product, and nothing in the system objects, because from the interface's perspective the fabricated content is indistinguishable from genuine content.

The grounded variant inverts the relationship. Interface components are bound to tool results, not to model text. A product card exists if and only if a tool call returned a product; the model can request that a card be shown, but it cannot supply the content of that card. The consequence is a structural guarantee rather than a behavioural one: the class of error in which an agent recommends a product that does not exist is eliminated by construction rather than reduced by instruction. This principle is the central design commitment of the present project and is described in detail in Section 4.4.

2.4 Evaluating Agents: Output versus Behaviour

Evaluation practice for conversational systems has historically focused on output: is the reply fluent, relevant and helpful. For agentic systems this is insufficient and can be actively misleading, because an agent may produce entirely plausible output by an incorrect route. An agent that answers a comparison question from memory rather than by retrieving and comparing products may read perfectly well while being exactly the failure the architecture exists to prevent.

Behavioural evaluation instead asserts on the trajectory: which tools were invoked, in what order, and with what arguments. Because the decision policy in this project is expressed as explicit rules, those rules can be restated directly as executable assertions. A request judged too broad must produce a clarification action before any search; a request to identify the cheapest seller must produce a seller comparison rather than a substitute web search. The evaluation reported in Section 4.3 follows this approach, and Section 4.3 also reports two cases in which it detected failures that output review would have passed.

2.5 Technology Survey

The following technologies were selected for the implementation. Each choice is recorded with the reason it was preferred over the alternatives considered.

Layer	Technology	Rationale
Frontend framework	Next.js App Router, React	Server components reduce client bundle; file-system routing suits a small surface.
Interface library	Tailwind CSS, shadcn/ui, Radix	Accessible primitives with keyboard support; consistent component composition.
Model orchestration	Vercel AI SDK	Provides streaming, multi-step tool loops and typed tool definitions in one abstraction.
Agent service	Node.js, TypeScript (strict)	Shares types with the frontend; strict mode catches contract drift at compile time.
Schema validation	Zod	Validates tool arguments before any external call is made.
Catalogue access	Shopify Catalog MCP	Cross-merchant inventory with genuine purchase routes; the alternative was limited to one store.
Model access	OpenRouter, DeepSeek	Twenty-nine free-tier models, twenty-two tool-capable, selectable at run time.
Persistence	PostgreSQL with an ORM	Relational integrity for users, chats, messages and votes; JSON column for message parts.
Caching	Redis	Short-lived caching of browsable feed and web evidence; catalogue results are never cached.
Web evidence	Playwright	Browser automation for evidence retrieval without a commercial search subscription.
Testing	Vitest, Playwright, custom harness	Unit, browser and behavioural layers respectively.

Table 2.1: Technology selection and rationale

Chapter 3

Problem Statement / Requirement Specifications

The problem is to help a buyer move from an imprecise natural-language request to a small, defensible set of purchasable products, without hallucinated catalogue facts. A solution must handle ambiguity in the request, live cross-merchant search, follow-up refinement, product and seller comparison, external research evidence, streamed responses, durable persistence and safe external links, while remaining usable under the rate limits of free-tier model access.

3.1 Project Planning

Development was planned around a single vertical shopping journey rather than around horizontal layers, so that an end-to-end path existed from the earliest stage and could be exercised continuously. The sequence of milestones was as follows.

1. Establish catalogue integration and normalise the returned records into a single internal product and variant shape.
2. Define a track-agnostic agent registry so that additional specialised agents could be added later without altering routing or persistence.
3. Implement the discovery tools: search, refinement, pagination and display.
4. Implement the analysis tools: product detail, product comparison and seller comparison.
5. Render every tool output as a native interface component, establishing the grounding contract.
6. Add authentication, guest access and message persistence.
7. Build the non-conversational discovery surface for browsing without model cost.
8. Harden failure handling across catalogue errors, provider limits and dropped streams.
9. Verify the system through unit, browser, smoke and behavioural scenario tests.

An earlier approach based on a single seeded development store and a CSV import pipeline was evaluated and abandoned once the cross-merchant catalogue proved viable, because the seeded approach could not support genuine seller comparison. The earlier pipeline was retained in the repository for reference only and forms no part of the runtime.

3.2 Requirements Analysis (SRS)

3.2.1 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall accept a shopping intent expressed in natural language.	High
FR-02	The system shall ask at most one clarifying question when a missing constraint would materially change the result set.	High
FR-03	The system shall search live cross-merchant catalogue data with optional price, availability and destination filters.	High
FR-04	The system shall refine a previous search by merging newly supplied constraints rather than starting afresh.	High

FR-05	The system shall paginate additional results while excluding items already shown.	Medium
FR-06	The system shall render product results as structured cards derived from tool output.	High
FR-07	The system shall compare between two and four products in a native comparison view.	High
FR-08	The system shall list all sellers of a chosen product ordered by price.	High
FR-09	The system shall retrieve external web evidence for research and compatibility questions.	Medium
FR-10	The system shall present a purchase route to the merchant once an item is selected.	High
FR-11	The system shall persist conversations and permit their later retrieval.	High
FR-12	The system shall support guest access without registration.	Medium
FR-13	The system shall permit model selection at run time.	Low
FR-14	The system shall provide a non-conversational browsing surface over the same catalogue.	Low

Table 3.1: Functional requirements

3.2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Correctness	Every product claim presented to the buyer shall originate in a tool result from the current turn.
NFR-02	Correctness	The agent shall decline to answer a compatibility question when external evidence is unavailable.
NFR-03	Reliability	A dropped response stream shall be recoverable without loss of the buyer's message.
NFR-04	Reliability	An empty or off-target search shall trigger re-query before failure is reported.
NFR-05	Determinism	A given conversation shall produce identical output when reloaded.
NFR-06	Security	Credentials for external services shall reside in a single process.
NFR-07	Security	Requests to the agent service shall be authenticated at the process boundary.
NFR-08	Security	Passwords shall be stored only as salted hashes.
NFR-09	Usability	A demonstration path shall exist that requires no registration.
NFR-10	Usability	The interface shall be responsive and keyboard accessible.
NFR-11	Maintainability	Tool arguments shall be schema-validated before external calls.
NFR-12	Compliance	Catalogue search results and images shall not be cached.

Table 3.2: Non-functional requirements

3.3 System Design

3.3.1 Design Constraints

The implementation requires Node.js 20 or later, a modern browser, Docker for the local datastores, environment configuration for both processes, and outbound HTTPS access to the catalogue and the selected model provider. Four constraints shaped the design materially.

- Free-tier model access introduces rate-limit variability and weaker instruction following. Both effects are visible in the evaluation results and are reported rather than suppressed.
- Merchant catalogue records are inconsistently populated. Specification and rating fields are frequently absent, so the interface hides empty rows rather than synthesising values.
- Currency values are not normalised across merchants, so comparison views display each variant in its native currency.
- Web evidence depends on third-party page markup and is therefore fragile by nature.

These constraints are handled through explicit fallbacks and visible limitations rather than through fabricated data, which is consistent with the grounding commitment described in Section 2.3.

3.3.2 System Architecture

The system comprises three cooperating processes and two datastores. The separation is not decorative: the agent service is the only component that holds credentials for the catalogue, the language models or web search. The web layer never contacts those services directly and instead forwards requests across an authenticated internal boundary, so the credential surface is confined to a single process.

Two independent checks protect that boundary. A shared secret authenticates the calling process, ensuring that only the application's own web layer can drive the agent and consume inference budget. Separately, every request carries a user identity, and request handlers verify that the requesting user owns the conversation before serving it. Guest sessions are real anonymous records, so guests and registered users traverse identical code paths.

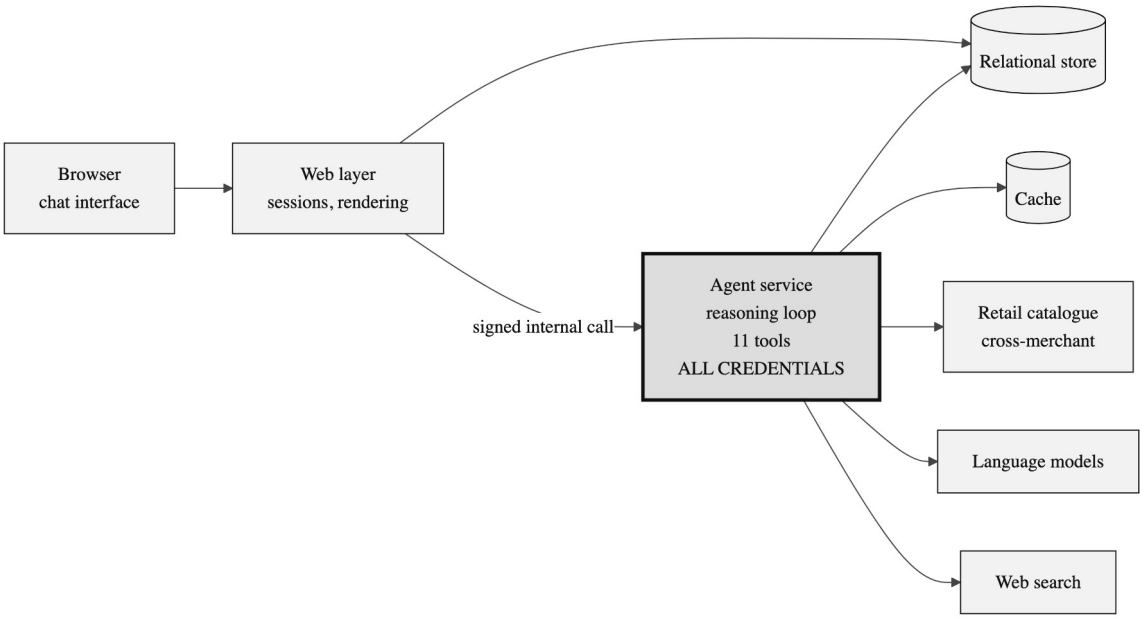

Figure 3.1: System architecture and trust boundary

3.3.3 Request Lifecycle

A single turn proceeds as shown in Figure 3.2. The ordering of the two persistence operations is deliberate: the buyer's message is committed to storage before the response stream opens, so a dropped connection can never lose their input. Working memory is rebuilt from the stored conversation at the start of every request, the agent is constructed for that specific conversation, and the reasoning loop then runs until a stopping condition is met. The model never observes raw network traffic; it observes typed tool results and selects its next action from them.

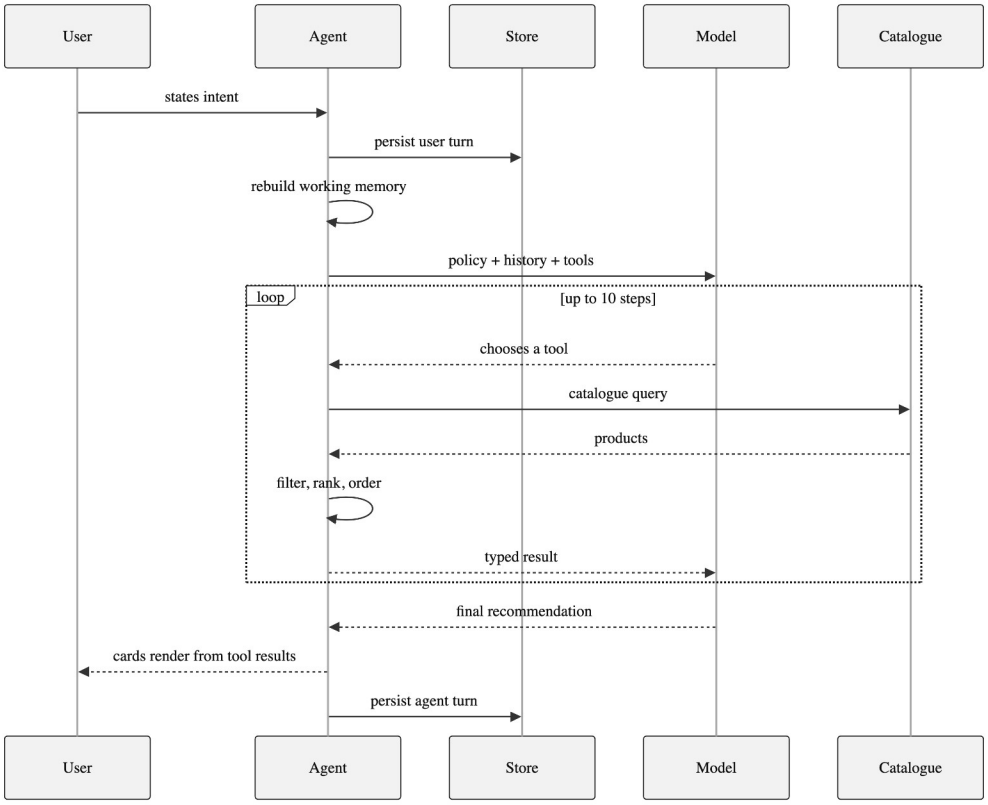

Figure 3.2: Request lifecycle for a single turn

3.3.4 Data Model and Caching

Persistence uses a relational store accessed through an ORM from both processes. Four entities are relevant to the shopping agent: users, conversations, messages and votes. The significant design decision is the storage of each message as an ordered collection of parts within a single JSON column. That collection records the assistant's text, its reasoning trace, and every tool call together with its input and its output.

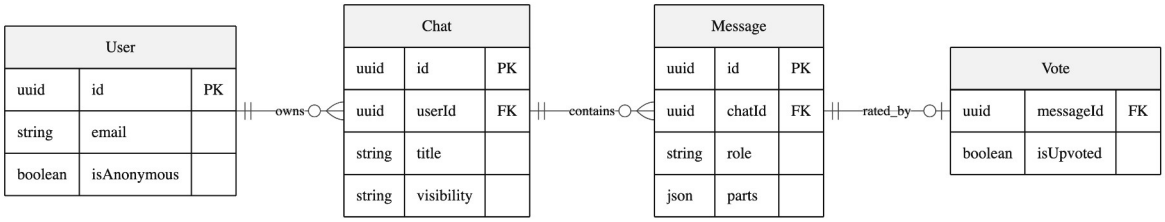

Figure 3.3: Persistence model

That single decision is what makes three further capabilities possible. Historical conversations re-render exactly as they originally appeared, because the tool results needed to reconstruct the interface are still present. Working memory can be rebuilt from the transcript rather than maintained as separate state, which removes an entire class of bug in which cached state and conversation history disagree. Finally, the evaluation harness can score a turn by reading the sequence of actions recorded within it.

Caching is applied only where it is both safe and useful. Browsable feed results are cached for one hour and feed searches for five minutes, because that surface is deterministic and carries no model cost. Web evidence is cached for five minutes within the running process;

this is more consequential than it appears, since without it two evidence lookups in a single turn would each launch a separate browser instance. Catalogue queries issued by the agent are never cached, both because freshness is the entire purpose of using a live catalogue and because the catalogue's terms prohibit it.

Chapter 4

Implementation

Implementation follows a tool-oriented architecture. Each shopping capability has a typed input schema and a narrow responsibility, and the frontend maps completed tool parts to specific interface components. The agent exposes eleven tools, constructed per conversation so that concurrent conversations cannot leak state into one another.

4.1 Methodology

4.1.1 The Reasoning Loop and Its Stopping Conditions

Each turn permits the model up to ten reasoning steps, after which the turn ends regardless of its state. This budget bounds cost and prevents a confused model from looping indefinitely on a failing search.

The second stopping condition is more interesting and is the more consequential of the two. When the agent asks the shopper a clarifying question, the turn ends immediately. Without this rule the model would ask a question and then continue working, rendering a product carousel underneath its own unanswered question; both the interface and the buyer found that outcome incoherent during development. Making clarification terminal converts a conversational convention into a structural guarantee, and it is enforced by the runtime rather than requested in the prompt.

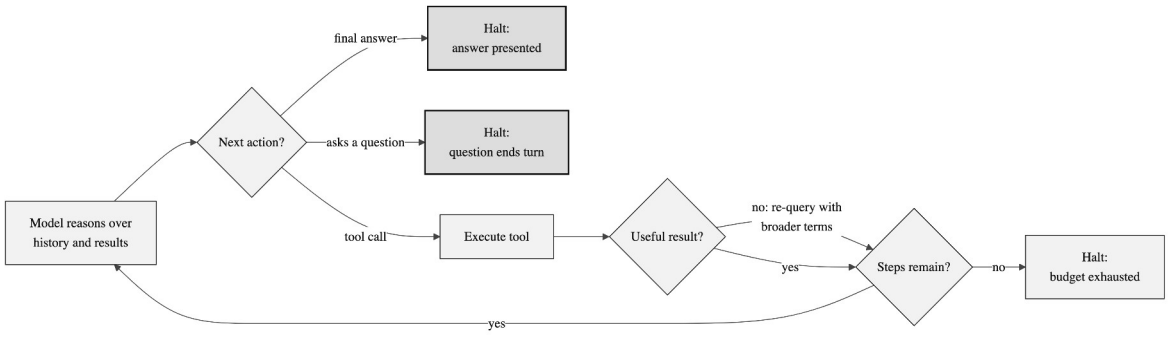

Figure 4.1: The agent reasoning loop and its three exits

The self-correction edge shown in Figure 4.1 is policy-driven. On an empty or off-target result the agent is required to re-query with broader or alternative terms, and may report failure to the shopper only after several differentiated attempts have not succeeded.

4.1.2 Decision Policy

The agent's planning policy is a decision tree over the shape of the buyer's utterance, written as explicit and testable rules rather than as general encouragement. Because the rules are explicit, they can be restated directly as assertions in the evaluation harness, which is what makes the behavioural evaluation in Section 4.3 possible.

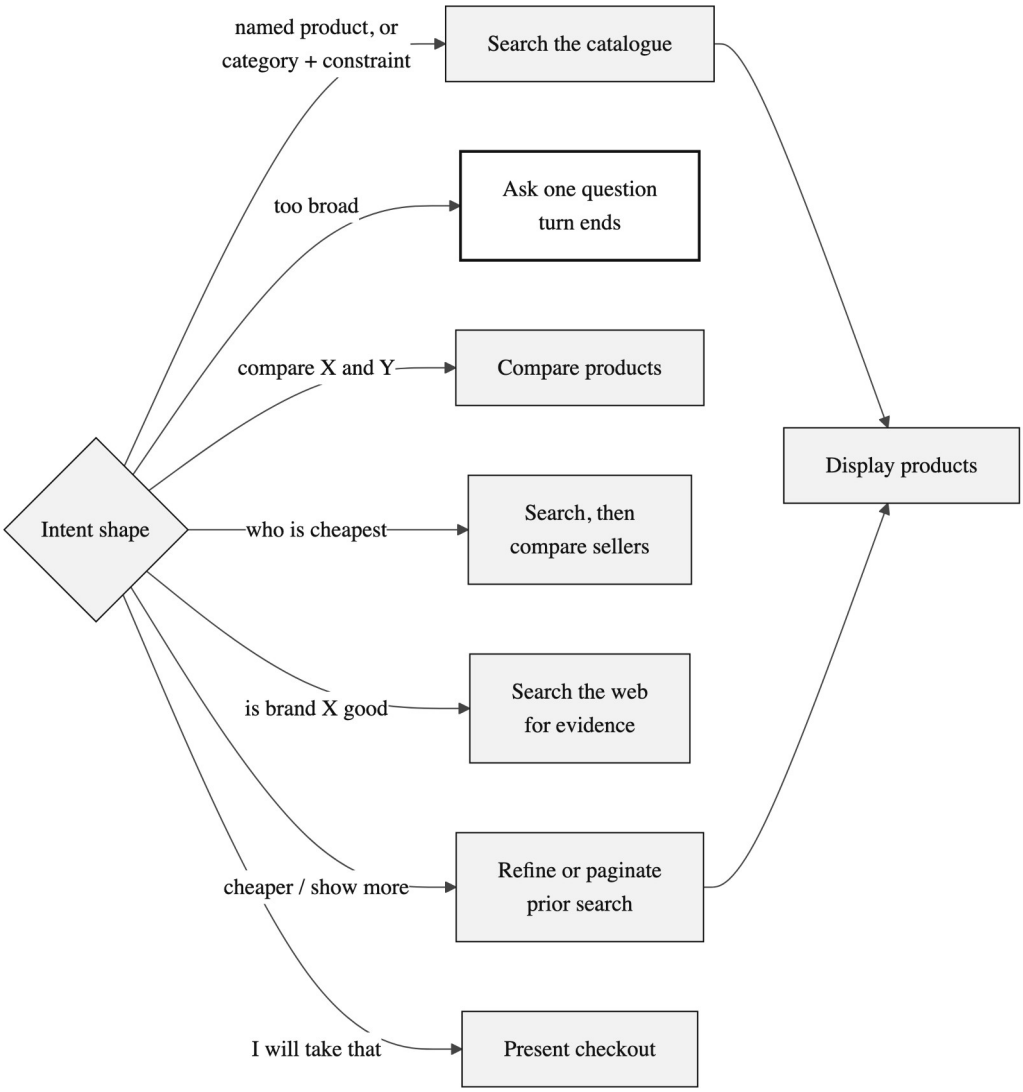

Figure 4.2: Intent routing decision policy

Only the clarification branch is terminal. The search-shaped branches converge on the single action capable of placing a product card on screen, which is the mechanism that enforces the separation described in the next subsection.

4.1.3 Tool Design

The eleven tools divide into four groups by responsibility. Every tool declares a typed and validated input schema, so malformed calls are rejected before any external request is issued.

Group	Tools	Responsibility
Discovery	Search, Refine, Show more	Query the catalogue. Refine narrows the prior query by merging new constraints; show-more paginates it while excluding items already seen.
Presentation	Display products	The only action capable of placing a product card on screen.
Analysis	Get product, Compare products, Compare sellers	Detail for one item, side-by-side comparison of two to four items, and every merchant offering a given product ordered by price.

Interaction and evidence	Clarify intent, Buy, Web search, Web fetch	Ask a multiple-choice question, present a purchase route, and gather external evidence for questions the catalogue cannot answer.

Table 4.1: Tool groups and responsibilities

Searching and displaying are deliberately distinct actions. Search results are private working data for the agent to reason over; placing them in front of the buyer requires a second, explicit decision. This separation is what allows the agent to search three times, discard two result sets as off-target, and present only the third, rather than showing the buyer every attempt it made.

4.1.4 Ranking Pipeline

The search tool does more than relay the catalogue response. Between request and result it applies a four-stage pipeline, each stage of which is deterministic given the conversation seed.

Stage	Operation	Purpose
1	Overscan	Request more products than will be shown, so later stages have slack to discard from.
2	Rating filter	Remove products rated below the threshold, falling back to the unfiltered set if too few survive.
3	Rerank	Apply the requested ordering: relevance, ascending price, descending price, rating or random.
4	Seeded shuffle	In relevance mode, hold the strongest results in place and lightly vary the order of the remainder.

Table 4.2: Stages of the ranking pipeline

The rating filter exists because of an observed failure rather than a hypothetical one. A very poorly rated product ranked first by pure catalogue relevance made the entire agent appear broken during testing, even though the ordering was technically correct. The fallback clause exists because the first version of that filter produced empty result sets in thinly populated categories, replacing one visible defect with another.

4.1.5 Working Memory

Follow-up requests such as “cheaper” or “show me more” are meaningless without state. The agent maintains a compact record of the last query issued, the filters applied, and the identifiers already shown, so that a refinement narrows the previous search rather than starting a new one, and pagination excludes what the buyer has already seen.

The design decision worth defending is that this memory is never stored separately. It is reconstructed from the conversation history at the start of every request. Because each tool result is itself recorded as part of the conversation, as described in Section 3.3.4, the memory is fully recoverable from the transcript. This eliminates the class of bug in which separately cached state and conversation history disagree, at the cost of a small amount of reconstruction work per request.

4.1.6 Reproducibility with Variation

A demonstration in which two evaluators type the same query and receive byte-identical output appears scripted, whereas one in which the same conversation yields different answers

on reload appears broken. The system resolves this tension by deriving a numeric seed from the conversation identifier and using it to vary three presentational elements: the tone hint supplied to the model, the ordering of lower-ranked search results, and the suggested-action copy shown on an empty conversation.

The resulting property is that the same conversation always produces the same output, while different conversations differ in voice but never in fact. Variation is confined strictly to presentation and never touches which products exist or what they cost.

4.2 Testing and Verification Plan

Verification combines deterministic checks with live-agent scenarios, arranged in four layers so that each class of defect is caught by the cheapest mechanism capable of detecting it.

Layer	Mechanism	Coverage
Static	Strict type checking	Compile-time contracts between the agent service, the tools and the persistence layer.
Unit	Backend test suite	Routing, authentication enforcement, history, messages, votes, uploads and server actions.
Browser	End-to-end specifications	Chat input, interface integration, authentication screens, suggested actions, generation stopping and model selection.
Behavioural	Scenario harness	Eight shopping scenarios scored on tool order, response text, failures and provider limits, with archived transcripts.

Table 4.3: Layers of the verification strategy

Representative test cases from the unit and browser layers are given in Table 4.4.

Test	Condition	System behaviour	Expected result
T01	Request without user headers	Reject request	Unauthorised response
T02	Request for a chat owned by another user	Ownership check fails	Forbidden response
T03	Category plus one constraint	Catalogue search executes	Product cards render
T04	Two product identifiers supplied	Comparison tool executes	Native comparison view
T05	Cheapest-seller request	Seller comparison executes	Sellers ordered by price
T06	Follow-up constraint after a search	Refinement merges with memory	Narrowed result set
T07	Repeat request for more options	Pagination excludes seen items	New products only
T08	Stream interrupted mid-response	Auto-resume path invoked	Conversation recovers
T09	Model emits a markdown table	Defensive parser strips it	Native view only
T10	Model wraps reply in an envelope	Parser unwraps inner text	Clean prose rendered

Table 4.4: Representative test cases

4.2.1 Behavioural Scenario Harness

The behavioural layer is the one most specific to an agentic system and warrants a fuller description. The harness drives the real system through eight scenarios, each chosen to exercise a distinct branch of the decision policy shown in Figure 4.2. Every scenario begins a fresh conversation, so no state carries between them. The harness captures the full response

stream, extracts the sequence of actions taken, and applies assertions derived directly from the agent's own policy.

One methodological rule matters more than the others. When a provider rate limit occurs during a run, the behavioural assertions are suppressed for that run and the limit is recorded instead. Without this rule, ordinary infrastructure flakiness would be recorded as a reasoning defect, and the results would be meaningless. Distinguishing infrastructure failure from reasoning failure is a precondition for the results being interpretable at all.

4.3 Result Analysis

The implemented system presents a responsive chat interface with guest access, conversation history, rotating suggestions, an in-conversation model selector and a link to the discovery surface. Static type checking passes without error and the backend unit suite passes in full. The behavioural results are reported below.

Scenario tests	Request	Outcome
Named product	find a specific pair of earbuds	Clean
Budget constraint	running shoes under a stated price	Clean
Comparison branch	compare two e-readers	Clean
Seller comparison	cheapest place to buy a named mouse	Clean
Category plus use case	laptop for video editing under a budget	Clean
Broad request	I need new headphones	Question emitted as text
Open-ended gift	gift for someone who likes cooking	Question emitted as text
Generation sensitivity	gaming phone under a stated price	Off-target results

Table 4.5: Behavioural evaluation results

Five of the eight scenarios ran cleanly and three were flagged. The failures are informative rather than incidental, and each has a diagnosed cause.

In the two broad-request scenarios the model produced a correctly structured clarifying question but wrote it as ordinary text instead of invoking the clarification action. The interface's defensive parsing meant that the buyer still saw a working option menu, so the visible outcome was correct, but the agent's action sequence was wrong. This is a limit of free-tier instruction following; the appropriate remedy is constrained decoding or a stronger model, not additional prompt text.

In the third flagged scenario the catalogue returned accessories rather than devices. The agent correctly recognised the results as off-target and said so rather than presenting wrong products, which is the behaviour the policy requires, but it did not exhaust its retry budget before doing so.

The finding of principal interest is that two of the three failures produced output that appeared entirely correct on screen. Had the system been evaluated by reading its replies, as conversational systems conventionally are, both would have been recorded as successes. Only assertions over the action sequence detected them. This supports the argument advanced in Section 2.4: for agentic systems, behavioural evaluation is not a supplement to output review but the primary instrument.

4.4 Quality Assurance

4.4.1 The Grounding Contract

Quality assurance rests first on the grounding contract, which is the mechanism by which fabricated product claims are made impossible rather than merely unlikely. Two paths exist for model output, and only one of them can produce a product card.

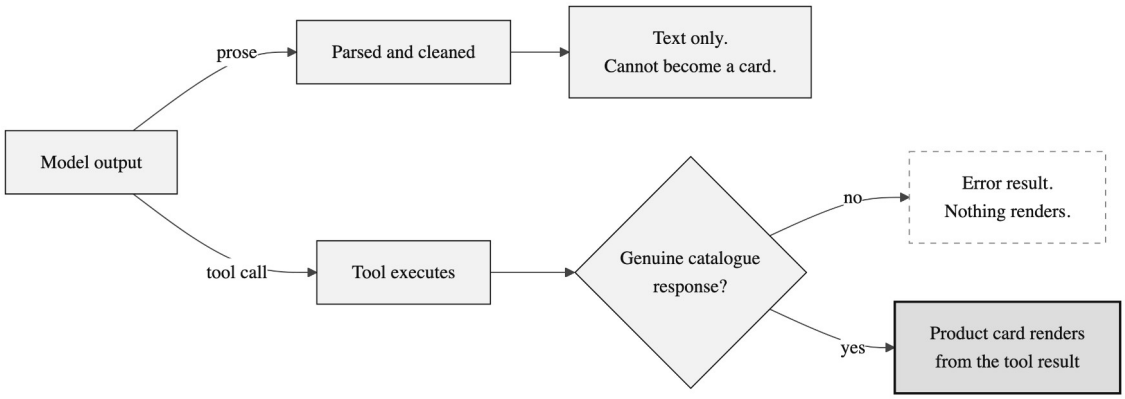

Figure 4.3: The grounded generative UI contract

The upper path in Figure 4.3 is the one the model fully controls, and it terminates in text. The lower path requires a genuine catalogue response before any card is rendered; if the tool call fails, the result is an error state and nothing is displayed. The model can request that a card be shown but cannot supply its content.

4.4.2 Defensive Parsing

The structural guarantee holds, but weaker models still violate the formatting contract frequently. Rather than assuming that prompt engineering had solved this, the interface assumes the model will misbehave and defends against it in three ways: it unwraps structured responses even when the model surrounds them with additional prose; it removes duplicate markdown tables that the model was instructed not to produce; and it recovers a question menu even when the model writes it as text rather than invoking the proper action.

The third defence is a workaround rather than a fix, and is reported as such. It is the mechanism that allowed the two flagged scenarios in Section 4.3 to appear correct on screen. Reporting those runs as successes because the interface recovered would have concealed a genuine behavioural defect.

4.4.3 Failure Handling

Failure	Detection	Response
Catalogue returns nothing or off-target results	Tool result inspection	Re-query within the same turn using broader or alternative terms before reporting to the buyer.
Catalogue transient error	Non-success response from the catalogue	Surfaced as a tool error the agent can react to; typically resolves on retry.
Provider rate limit	Error raised during streaming	Surfaced to the buyer; recovery is by manual model selection.
Response stream drops	Client-side timeout	Synthetic timeout message and retry affordance.
Conversation opens on an unanswered message	Stored history inspection	Stream resumption by replay rather than re-execution.

Interface state diverges from storage	Stream completion handler	Revalidate and replace in-memory state with the stored snapshot.
Duplicate comparison invocations in one turn	Message part inspection	Only the final comparison view is rendered.
Comparison rows with no data in any column	Comparison component	Row hidden entirely rather than filled with placeholder characters.
Primary search engine blocked	Zero parsed results	Fall through to the secondary engine.
Poorly rated product ranked first	Ranking pipeline stage 2	Removed below threshold, with an unfiltered fallback for sparse categories.

Table 4.6: Failure handling matrix

Errors are surfaced without inventing results. Interrupted conversations can resume, a client timeout provides recovery guidance, external links are validated before rendering, and malformed model prose is stripped of envelopes, procedural narration and duplicate tables before display.

Chapter 5

Standards Adopted

5.1 Design Standards

The interface follows responsive web design, consistent component composition, keyboard-accessible primitives, and progressive disclosure of detail. Product facts are presented in cards and comparison matrices while assistant prose remains short, so that the structured evidence rather than the narration carries the factual load.

The system separates browser, web layer, agent service, storage, catalogue and model provider responsibilities along process boundaries. Interface boundaries use typed JSON with schema validation and explicit message-part states, so that a partially completed tool call is representable and renderable rather than being an undefined condition.

5.2 Coding Standards

The project follows strict static typing and repository-level formatting, naming, validation and security conventions.

- Use strict types and schema validation at every tool and request boundary.
- Use descriptive camelCase function names, PascalCase component names, and domain-specific type names that reflect catalogue vocabulary.
- Keep each shopping tool focused on a single operation and reuse the normalised catalogue models rather than re-deriving shapes per tool.
- Keep credentials in ignored environment files and protect internal service calls with a shared secret.
- Run formatting and type checks and remove debug output before delivery.
- Use non-caching requests for live catalogue data and validate every external merchant link before rendering it.

5.3 Testing Standards

Testing follows the layered strategy set out in Table 4.3: unit and route behaviour at the static and unit layers, browser workflows for the principal shopping journey, and scenario-based agent evaluation focused on tool sequencing and grounding indicators.

One standard is specific to this project and is recorded deliberately. Failures and provider limits are preserved in transcript artifacts rather than being hidden or reclassified as passes. An evaluation that quietly discards inconvenient runs cannot support any claim about the system's behaviour, and the value of the harness described in Section 4.2.1 depends entirely on this discipline being maintained.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

AI Shopping Agent demonstrates a practical architecture for conversational commerce grounded in live merchant data. It reduces search friction by translating natural-language intent into a sequence of clarification, catalogue search, refinement, comparison, seller analysis, evidence lookup and purchase handoff operations, each bounded by explicit stopping conditions and each producing typed results the interface can render directly.

The strongest design result is the grounding contract. Product cards and comparison surfaces can appear only from completed tool outputs, which means the agent is incapable of presenting a product it did not retrieve. Combined with strict schemas, persistence synchronisation, link validation and defensive rendering, this makes the system substantially more trustworthy than a prose-only shopping assistant, and it does so by construction rather than by instruction.

The evaluation supports a more specific and more transferable claim. Two scenarios produced output that looked entirely correct on screen while the agent's underlying behaviour was wrong, and only assertions over action sequences detected them. For agentic systems this suggests that behavioural evaluation is not an optional addition to output review but the primary instrument, and that a system's honesty about its own failure modes is a better indicator of engineering quality than a demonstration in which nothing goes wrong.

6.2 Future Scope

Three improvements follow directly from the limitations observed during evaluation and are ordered by the value they would add.

1. Automatic failover between models. The metadata required to select an alternative model already exists but is not consulted at run time. Retrying a rate-limited request against the next available model would close the most visible operational gap in the current system.
2. Constrained output for clarifying questions. Forcing the model to emit a structured action rather than relying on its compliance would fix both flagged evaluation scenarios at the root, instead of recovering from them in the interface.
3. A larger evaluation set. The present eight scenarios cover the main branches of the decision policy but not multi-turn refinement chains, which is precisely where working memory is exercised and where regressions would be least visible.

Beyond these, further work should add multi-currency normalisation in comparison views, saved carts and cross-conversation preferences, richer accessibility testing, merchant-specific catalogue support, and stronger observability for tool latency and failure rates. The agent registry already accommodates additional specialised agents for checkout recovery, checkout

assistance, customer support and merchant catalogue optimisation without altering the core routing shape.

Further evaluation should use a stable paid model baseline to separate policy defects from model defects, a larger catalogue-quality benchmark, automated link-health checks, and human scoring for relevance, trust, time-to-recommendation and purchase confidence. Native mobile clients and image-based similar-product search remain outside the present scope.

References

[1] Shopify, “Storefront MCP server and UCP catalog tools,” Shopify Developer Documentation, 2026.

[2] Anthropic, “Model Context Protocol specification,” MCP Documentation, 2026.

[3] Vercel, “AI SDK Core: streamText,” AI SDK Documentation, 2026.

[4] Vercel, “Next.js App Router,” Next.js Documentation, 2026.

[5] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” 2026.

[6] Redis, “Redis Documentation,” 2026.

[7] OpenRouter, “Quickstart Guide,” OpenRouter Documentation, 2026.

[8] Microsoft, “Playwright Test: Introduction,” Playwright Documentation, 2026.

[9] Colin Zod, “Zod: TypeScript-first schema validation,” Zod Documentation, 2026.

[10] Drizzle Team, “Drizzle ORM Documentation,” 2026.